IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

# De texto a números: qué opinan los clientes

Teoría, práctica y, al final, material de consulta

Sprint 12 · S12_03 · teoría breve (20 min el 23/11), 55 min de práctica en clase y material de consulta

LA IDEA · DEL TEXTO AL NÚMERO

# Un modelo no lee: calcula con números

### 1 · Reseña

```
¡El BAÑO estaba SUCIO!!!
```

El texto, tal cual lo escribe el cliente.

→

### 2 · Limpiar

```
el baño estaba sucio
```

Minúsculas y sin signos: una sola forma por palabra.

→

### 3 · Trocear

```
['baño', 'sucio']
```

**Tokens** (palabras), sin las palabras vacías.

→

### 4 · Contar

```
baño 1 · sucio 1
```

Un número por cada palabra del vocabulario.

→

### 5 · Clasificar

```
negativo
```

Un modelo, entrenado con reseñas, elige la etiqueta.

### Qué recibe el modelo

**Números**, no palabras: por eso el texto se convierte antes en una tabla de recuentos.

### Dos preguntas del cliente

¿Qué opina la gente? → **sentimiento**.

¿De qué habla? → **temas** (limpieza, ruido, desayuno…).

### Ojo: son reseñas sintéticas

Inventadas para el curso: más repetitivas y fáciles que las reales.

LA IDEA · LIMPIAR EL TEXTO

# Limpiar: «Bueno» y «BUENO» son la misma palabra

| Reseña tal cual | Después de **limpiar** |
|---|---|
| ¡El BAÑO estaba SUCIO!!! | el baño estaba sucio |
| Wifi: 0 estrellas... ¿en serio? | wifi estrellas en serio |
| Desayuno 10/10, ¡volveremos! | desayuno volveremos |

### Las tres reglas de la limpieza

1. Todo a minúsculas.
2. Lo que no sea letra ni espacio, por un espacio.
3. Un solo espacio entre palabras.

### Dos trampas

La **ñ** y las **tildes** son letras: «baño» y «ubicación» se quedan como están.

Un número o un símbolo (**0**, **10/10**) no es una letra: desaparece.

### Palabras vacías

«de», «el» o «la» casi no dicen nada: se quitan al trocear. «no» se **conserva**: cambia el sentido.

LA IDEA · CONTAR PALABRAS

# Contar palabras: lo más frecuente no es lo más útil

### Trocear y contar

**El baño estaba sucio, no me gustó** → **baño · sucio · no · gustó**.

Contar cada palabra: **wifi · cama · wifi** → **wifi** 2, **cama** 1.

### Mini-corpus de 4 reseñas

Wifi lento y cama dura

Cama cómoda y wifi gratis

Wifi caído otra vez

Piscina limpia y cama cómoda

| Las 3 palabras más frecuentes | Resultado |
|---|---|
| Con todas las palabras | **wifi** 3 · **y** 3 · **cama** 3 |
| Sin palabras vacías (PALABRAS_VACIAS) | **wifi** 3 · **cama** 3 · **cómoda** 2 |

**«y» se repite tanto como «wifi»; y «wifi» sale en 3 de 4 reseñas: frecuente no es lo mismo que informativo.**

LA IDEA · DEL RECUENTO AL PESO

# Bolsa de palabras y TF-IDF

| reseña | cama | cómoda | dura | el | es | la | lento | wifi |
|---|---|---|---|---|---|---|---|---|
| r1 · la cama es cómoda | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| r2 · la cama es dura | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| r3 · el wifi es lento | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |

| Palabras de r1 | Está en… | Bolsa | TF-IDF |
|---|---|---|---|
| es | 3 de 3 reseñas | 1 | 0,37 |
| la, cama | 2 de 3 | 1 | 0,48 |
| cómoda | 1 de 3 | 1 | **0,63** |

### Bolsa de palabras

Un recuento por palabra del vocabulario. Se pierde el orden.

### TF-IDF

Recuento × rareza: lo que está en **todas** las reseñas pesa menos.

LA IDEA · UN PIPELINE SIN FUGA

# Dos modelos con el mismo pipeline

**75 %**
**Entrenamiento**
aquí se aprende, vocabulario incluido

**25 %**
**Prueba**
solo para medir

### Un pipeline

**make_pipeline(vectorizador, clasificador)**.

**fit** con el **entrenamiento**; a la prueba solo **predict**.

### Sin fuga de datos

El vectorizador va **dentro** del pipeline: no «espía» la prueba.

### Regresión logística + TF-IDF

Un peso por palabra y clase: suma los pesos de la reseña y elige la clase con más puntos.

### Naive Bayes + bolsa de palabras

Cuenta cuánto aparece cada palabra en cada clase y elige la clase más probable.

### Una referencia

Predecir siempre la clase más frecuente da una exactitud mínima: tus modelos deben **superarla**.

# La bolsa de palabras no ve el orden

| Dos frases con sentidos opuestos | Su bolsa de palabras |
|---|---|
| Está limpio, no está caro | **caro** 1 · **está** 2 · **limpio** 1 · **no** 1 |
| No está limpio, está caro | **caro** 1 · **está** 2 · **limpio** 1 · **no** 1 |

| Frase nueva para el modelo | ¿Qué predice? |
|---|---|
| El personal fue amabilísimo | ? |
| El personal **no** fue amabilísimo | ? |
| El desayuno es caro | ? |
| El desayuno **no** es caro | ? |

### Pregunta para el grupo

Las dos bolsas son **idénticas**. ¿Qué información de la frase se ha perdido?

¿Cómo la guardarías? Y, con y sin «no», ¿qué etiqueta crees que dará el modelo?

LA IDEA · TEMAS POR PALABRAS CLAVE

# Temas por palabras clave: transparente y barato

### Un diccionario

Cada tema, una lista de claves: **wifi** → **wifi**, **internet**, **conexión**.

Si la reseña limpia **contiene** alguna, tiene el tema.

### F1 = 0,67

Mezcla dos medidas: de lo que **detecta**, cuánto es real (**2 de 3**); de lo real, cuánto encuentra (**2 de 3**).

| Reseña | ¿Habla de wifi? | ¿Lo detecta? | Resultado |
|---|---|---|---|
| El wifi no funciona | sí | sí | acierto |
| Internet va muy lento | sí | sí | acierto |
| La conexión con el aeropuerto es buena | no | sí | **falso positivo**: clave demasiado amplia |
| Cobran por conectarse a la red | sí | no | **falso negativo**: falta una clave |
| La piscina es enorme | no | no | acierto |

LA IDEA · N-GRAMAS

# n-gramas: palabras que van juntas

### n-grama = n palabras seguidas

«no compensa el precio» tiene 3 **bigramas** (**no compensa**, **compensa el**, **el precio**) y 2 **trigramas**.

### Para qué sirven

Conservan parte del **orden**: «no compensa» dice más que «no» y «compensa» por separado.

| n-grama de «no compensa el precio» | palabras |
|---|---|
| no compensa | 2 |
| compensa el | 2 |
| el precio | 2 |
| no compensa el | 3 |
| compensa el precio | 3 |

**Un trigrama contiene bigramas más cortos: al contar n-gramas, los mismos casos aparecen varias veces.**

PRÁCTICA · CUADERNO S12_03

# Lo que haremos en el cuaderno

| Parte | Qué haremos | Dónde |
|---|---|---|
| **Preparación** | Cargar las reseñas y ver el sentimiento | Clase |
| **Ejercicio 1** | Limpiar un texto y quitar palabras vacías | Clase |
| **Ejercicio 2** | Las 5 palabras más frecuentes | Clase |
| **Ejemplo** | Bolsa de palabras, TF-IDF y partición | Clase |
| **Ejercicio 3** | Regresión logística frente a Naive Bayes | Clase |
| **Ejemplo** | Informe, matriz, pesos y negaciones | Clase |
| **Ejercicio 4** | ¿Falla con las negaciones? | Clase |
| **Ejemplo** | Temas por palabras clave y F1 | Clase |
| **Ejercicio 5** | El tema peor y dónde falla; n-gramas | Casa |
| **Mini-reto** | Panel de quejas por tema | Casa |

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el** ✔. La solución está plegada: ábrela tras intentarlo.

CONSULTA

# Material de consulta: código y aclaraciones

### Cuándo abrirlo

**Después de intentar el ejercicio**, o cuando algo no te salga. En clase no se proyecta.

### Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

| Ejercicio | Tema | Diapositivas |
|---|---|---|
| **1** | Limpiar un texto | 12 y 13 |
| **2** | Las 5 palabras más frecuentes | 14 y 15 |
| **3** | Logística frente a Naive Bayes | 16 y 17 |
| **Ejemplo** | Resultados, pesos y negaciones | 18 y 19 |
| **4** | ¿Falla con las negaciones? | 20 y 21 |
| **Ejemplo** | Temas por palabras clave y F1 | 22 |
| **5** | El tema peor y los n-gramas | 23, 24 y 25 |
| **Mini-reto** | Panel de quejas | 26 y 27 |

CONSULTA · EJERCICIO 1

# Limpiar un texto

```
def limpiar(texto):
    texto = texto.lower()
    texto = re.sub(r"[^a-záéíóúüñ\s]", " ", texto)
    return " ".join(texto.split())
```

**qué bueno sí**

limpiar("¡Qué BUENO!... Sí")

**precio ok**

limpiar("Precio 3€, ok")

Las tildes y la ñ se conservan: son letras del español.

CONSULTA · ACLARACIONES · EJERCICIO 1

# Leer la limpieza (ejercicio 1)

**[^a-záéíóúüñ\s]**

Todo lo que no sea letra (con tildes y ñ) ni espacio se cambia por un **espacio**.

**" ".join(texto.split())**

Colapsa los espacios repetidos en **uno solo**.

## Resultado

**limpiar("¡Qué BUENO!... Sí")** da **«qué bueno sí»**.

## Cuidado

Sin **áéíóúüñ**, «baño» quedaría como «ba o»: la expresión debe **conservarlas**.

CONSULTA · EJERCICIO 2

# Las 5 palabras más frecuentes

```
from collections import Counter
contador = Counter(t for texto in resenas["texto"] for t in tokenizar(texto))
top5 = contador.most_common(5)
```

**semana · 504**

Más frecuente

**desayuno 391 · habitación 284**

Otras con contenido

«semana», «todo» y «escapada» son relleno de frases de apertura: **frecuente no es informativo**.

CONSULTA · ACLARACIONES · EJERCICIO 2

# Leer las frecuencias (ejercicio 2)

### tokenizar(texto)

Limpia y quita las **palabras vacías**: «de» o «la» no llenan el recuento.

### Counter(...).most_common(5)

Cuenta los tokens y devuelve **5 pares (palabra, veces)** de mayor a menor.

### Resultado

**semana** 504 · **desayuno** 391 · **todo** 382 · **habitación** 284 · **escapada** 256.

### Cómo leerlo

«semana», «todo» y «escapada» vienen de **frases de apertura**; solo «desayuno» y «habitación» hablan de la estancia.

CONSULTA · EJERCICIO 3

# Regresión logística frente a Naive Bayes

```
modelo_lr = make_pipeline(TfidfVectorizer(preprocessor=limpiar), LogisticRegression(max_iter=1000)).fit(
    X_ent, y_ent)
modelo_nb = make_pipeline(CountVectorizer(preprocessor=limpiar), MultinomialNB()).fit(X_ent, y_ent)
acc_lr = accuracy_score(y_test, modelo_lr.predict(X_test))
acc_nb = accuracy_score(y_test, modelo_nb.predict(X_test))
```

**0,992**

Regresión logística + TF-IDF

**0,989**

Naive Bayes + bolsa de palabras

**Referencia: siempre «positivo» acierta 0,469. Un 99 % es sospechoso: reseñas sintéticas.**

CONSULTA · ACLARACIONES · EJERCICIO 3

# Leer los dos modelos (ejercicio 3)

### make_pipeline(vectorizador, clasificador)

El vectorizador va **dentro**: se ajusta solo con el entrenamiento, sin fuga.

### preprocessor=limpiar

Aplica **tu función de limpieza** a cada texto antes de contar palabras.

### Resultado

**0,992** (logística) y **0,989** (Naive Bayes) frente a **0,469** de la clase mayoritaria.

### Cuidado

Un 99 % no es creíble con reseñas reales: estas son **repetitivas y fáciles**.

CONSULTA · SECCIONES 4 Y 5 · INFORME Y PESOS

# Casi el 99 %: demasiado bonito para ser verdad

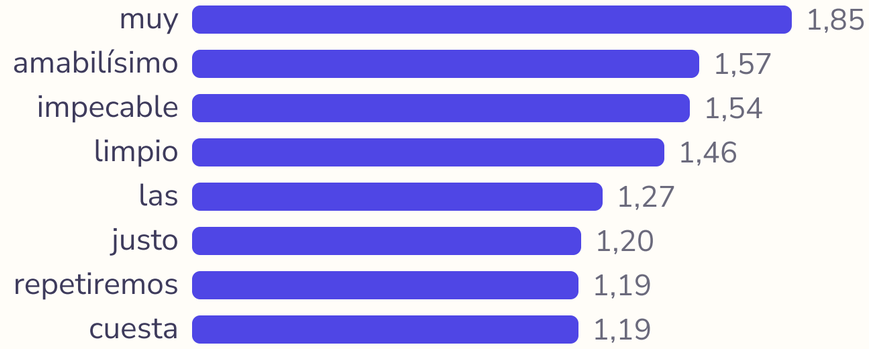
| Modelo (375 reseñas de prueba) | Exactitud |
|---|---|
| Siempre «positivo» | 0,469 |
| Regresión logística + TF-IDF | ≈ **0,99** |
| Naive Bayes + bolsa de palabras | ≈ 0,99 |

**Sospecha**

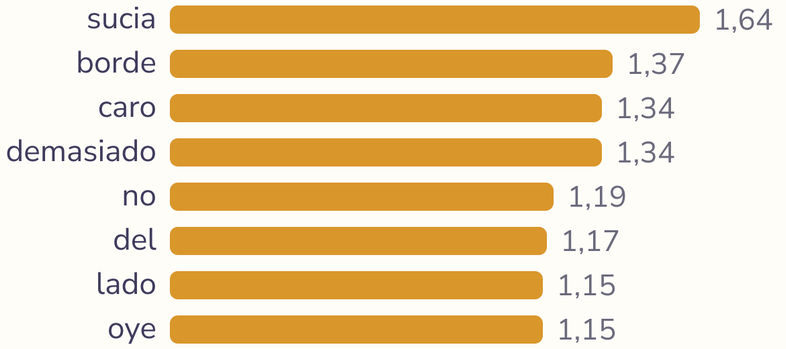
Fallan muy pocas reseñas, y las que fallan son neutras.

Con reseñas sintéticas no te fíes de este número.

**Empujan a «positivo»**




**Empujan a «negativo»**



# Probar frases con negaciones

| Dos frases con sentidos opuestos | Su bolsa de palabras |
|---|---|
| Está limpio, no está caro | caro 1 · está 2 · limpio 1 · no 1 |
| No está limpio, está caro | caro 1 · está 2 · limpio 1 · no 1 |

| Frase nueva | Predicción |
|---|---|
| La limpieza es impecable | positivo |
| La limpieza **no** es impecable | **positivo** (falla) |
| El personal fue borde | negativo |
| El personal **no** fue borde | **negativo** (falla) |

### Qué se ve

La regresión logística **ignora el «no»**: ve «impecable» o «borde» y se olvida de la negación.

Una bolsa de palabras no guarda el orden.

CONSULTA · EJERCICIO 4

# ¿Falla con las negaciones?

```
falla_negacion = modelo_lr.predict([pruebas[1]])[0] == "positivo"
bigrama_util = "Un bigrama junta dos palabras seguidas (p. ej. «no es»),
    así conserva parte del orden y de la negación."
```

**True**

falla_negacion

**positivo**

«La limpieza no es impecable»

Con **ngram_range=(1, 2)**, «no es» es una columna propia y el modelo puede distinguirlas.

CONSULTA · ACLARACIONES · EJERCICIO 4

# Leer las negaciones (ejercicio 4)

### predict([pruebas[1]])[0] == "positivo"

True si el modelo clasifica como **positiva** la frase con negación: falla.

### bigrama_util

Una explicación de **una frase**: un bigrama junta dos palabras seguidas y conserva el «no».

### Resultado

falla_negacion = **True**: «La limpieza no es impecable» sale **positivo**.

### Cómo arreglarlo

Con **bigramas** (ngram_range=(1, 2)) el «no es» es una pista propia, **si** el corpus contiene esa pareja.

CONSULTA · SECCIÓN 6 · TEMAS Y F1

# Temas por palabras clave y su F1

```
def detectar_temas(texto, diccionario=TEMAS):
    t = limpiar(texto)
    return sorted(tema for tema, claves in diccionario.items() if any(c in t for c in claves))
def f1_tema(tema):
    real = resenas["temas_reales"].apply(lambda l: tema in l)
    pred = resenas["temas_detectados"].apply(lambda l: tema in l)
    return f1_score(real, pred)
f1_por_tema = pd.Series({t: f1_tema(t) for t in TEMAS}).sort_values()
```

**0,94**

F1 de «ruido»

**1,00**

F1 de los otros cinco temas

**Transparente y barato: casi perfecto en este corpus sintético, salvo ruido.**

CONSULTA · EJERCICIO 5

# El tema peor detectado

```
tema_peor = f1_por_tema.idxmin()
n_precio = int(resenas["temas_reales"].apply(lambda l: "precio" in l).sum())
```

**ruido**

tema_peor

**455**

n_precio

El fallo: la clave **ruid** coincide con «ruidosa» y 55 reseñas se marcan como «ruido» sin serlo.

CONSULTA · ACLARACIONES · EJERCICIO 5

# Leer el tema peor (ejercicio 5)

### f1_por_tema.idxmin()

Devuelve el **nombre** del tema con el F1 más bajo; **min()** solo daría el valor.

### apply(lambda l: "precio" in l).sum()

Cuenta **reseñas** (no temas) cuya lista **temas_reales** contiene «precio».

### Resultado

**tema_peor** = **ruido** (F1 0,94) y **n_precio** = **455** de 1 500 reseñas.

### Remedio

Afinar el diccionario: cambiar la clave **ruid** (coincide con «ruidosa») por una más precisa.

CONSULTA · SECCIÓN 7 · N-GRAMAS DE LAS RESEÑAS NEGATIVAS

# Las quejas que una palabra suelta esconde

### Qué se ve

Las expresiones revelan **quejas concretas**: «ruido constante (de) noche» y «zona ruidosa y poco segura de noche».

### Por qué se repiten

Un trigrama **contiene** bigramas más cortos: los mismos casos se cuentan en cada uno.

| n-grama en reseñas negativas | veces |
|---|---|
| constante noche | 69 |
| ruido constante | 69 |
| ruido constante noche | 69 |
| ruidosa poco | 61 |
| ruidosa poco segura | 61 |
| poco segura | 61 |
| poco segura noche | 61 |
| zona ruidosa | 61 |

**Ocho filas, solo dos quejas repetidas en trozos de distinta longitud: el ruido por la noche y la zona poco segura.**

CONSULTA · MINI-RETO

# Panel de quejas por tema

```
quejas = resenas.loc[resenas["sentimiento"] == "negativo", "temas_detectados"].explode().value_counts()
queja_principal = quejas.index[0]
fig, ax = plt.subplots()
quejas.sort_values().plot.barh(ax=ax, color="#c0463a")
ax.set_title("Reseñas negativas por tema"); ax.set_xlabel("Número de reseñas")
```

## ruido · 201

queja_principal

## limpieza 171 · ruido 155

Con etiquetas reales

**Ojo antes de fiarte:** el panel sobreestima el ruido por una clave demasiado amplia.

CONSULTA · ACLARACIONES · MINI-RETO

# Leer el panel de quejas

### explode()

Pasa de una lista de temas por reseña a **un tema por fila**.

### value_counts()

Cuenta y **ordena de mayor a menor**: el primero es **queja_principal**.

### Resultado

ruido **201**, limpieza 171, precio 165, desayuno 162, ubicación 160 y personal 147.

### Validar

Con las etiquetas reales, limpieza (171) supera a ruido (**155**): un diccionario sin validar engaña.